

APMA E4300: Introduction to Numerical Methods

Michael Cai

April 1, 2025

Abstract

This course introduces and analyzes fundamental numerical methods in scientific computation. The main topics include elementary numerical linear algebra, least-square methods, nonlinear equations and optimization, polynomial interpolation, numerical differentiation and integration, initial-value and boundary problems for ordinary differential equations and systems, eigenvalue problems. This course prepares students for more advanced studies in the field of numerical analysis and scientific computation (for instance, APMA courses E4301, E4302, and E6302).

Contents

1	Linear Algebra Overview	2
1.1	Linear Iterative Schemes for Linear Systems	4
2	Interpolation	7
2.1	Lagrange Interpolation	8
2.2	Lagrange Interpolation Proof	8
2.3	Hermite Interpolation	9
2.4	Spline Interpolation	10
3	Numerical Differentiation	14
3.1	Definitions of Differentiation	14
3.2	General Approach $n + 1$ Point Formula	14
4	Numerical Differentiation	16
4.1	Definitions of Differentiation	16
4.2	General Approach $n + 1$ Point Formula	16
4.3	Richardson's Extrapolation	17

Chapter 1

Linear Algebra Overview

Lecture 1

Remark.

$$Av = \lambda v$$

When $\{v_i\}_{i=1}^n$ is linearly independent, then we will have that

$$Av_i = \lambda_i v_i$$

We can then write A as a product of two matrices such that we have

$$AQ = Q\alpha A = Q\alpha Q^{-1}$$

Remark. Suppose that some of the values do not contribute much in

$$A = Q\alpha Q^{-1}$$

Then we can throw away some nonzero elements given that they are small. Thus, we can create an approximation of the matrix which was a (256×256) matrix. We have converted it to a $n \times 2, 2 \times n$.

Example. We can have an image that is 256×256 , we can compress it without losing too much information. However, there is never any free lunch. Any two dimensional object, 3d, etc. can be done. Thus, we can generalize it further in the course...

With λ being very small, it is negligible and can be omitted.

We would like to know how much error can be omitted.

Definition 1.0.1. l^2 norm is going to be denoted as $\|\mathbf{x}\|$ which is a positive number given by

$$\sqrt{\sum_{i=1}^n x_i^2}$$

Theorem 1.0.1. The l_2 norm satisfies the following properties:

- $\|x\|_{l_2} = 0 \Leftrightarrow x = 0$
- $\|\alpha x\| = |\alpha| \|x\|_{l_2}$

Jan 23 10:10

Lecture 2

Jan 28 10:10

Remark. Gaussian Elimination:

Suppose we had an $n \times n$ system with n unknowns and n equations denoted as

$$Ax = y, A \in \mathbb{R}^{n \times n}$$

Example. Given an example of the equation

$$y = a_0 + a_1x + \dots + a_{m-1}x^{m-1}$$

How do we figure out the coefficients. Now suppose that we are given a set of points, it is quite easy to solve a matrix for the given system of equations

$$a_0 + a_1x + \dots + a_{m-1}x^{m-1} = y_1$$

...

$$a_0 + a_1x_m + \dots = y_m$$

This results in a matrix where we can solve for the values of the coefficients now.

$$\begin{pmatrix} 1 & x_1 & x_1^{m-1} \\ \dots & & \\ 1 & x_m & x_m^{m-1} \end{pmatrix} \begin{pmatrix} a_0 \\ \dots \\ a_m \end{pmatrix} = \begin{pmatrix} y_0 \\ \dots \\ y_m \end{pmatrix}$$

In practice, we could measure data points which leads to a situation where there are more equations or x values than coefficients which is defined as a "overdetermined matrix". On the other hand, we can also have an underdetermined problem.

Example. Suppose we have another example with a system of equations with $y = a_0 + a_1 \sin x + a_2 \sin 2x \dots a_n \sin x$. If we continue this this is also a linear system of equations where the matrix of the equations will be

$$\begin{pmatrix} 1 & \sin x_1 & \sin(m-1)x_1 \\ \dots & & \\ 1 & \sin x_m & \sin(m-1)x_m \end{pmatrix} \begin{pmatrix} a_0 \\ \dots \\ a_m \end{pmatrix} = \begin{pmatrix} y_0 \\ \dots \\ y_m \end{pmatrix}$$

One useful example is a gaussian mixture model where you can superimpose a number of gaussian models to get the value of y . Thus, solving $Ax = y$ gives many possible applications.

Elementary Row Operations:

Definition 1.0.2. Elementary row operations can be done with multiplication. Scaling is done with the following matrix

$$S_i(p) = \begin{pmatrix} 1 & & \\ & p & \\ & & 1 \end{pmatrix}$$

which gives us a scaling factor on a row of the matrix. We simply see that the inverse of the matrix would be simply be $\frac{1}{p}$ for $S_i(p)^{-1}$

Definition 1.0.3. Interchange: interchanging row i and row j of $\vec{A}$ gives us

$$E_{ij}A = \begin{pmatrix} 1 & & & \\ & 0 & 1 & \\ & 1 & 0 & \\ & & & 1 \\ & & & & 1 \end{pmatrix} A$$

This showcases a exchange of rows i and j . Note that the inverse of such a matrix is just itself.

Definition 1.0.4. Replacement is also a method too. (finish later)

Remark. Gaussian Elimination:

We can start with an easy example

$$A = \begin{pmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ \dots & \dots & \dots & \dots \end{pmatrix}$$

We can then apply a matrix L_1 which is our linear operator we use to transform our matrix using gaussian elimination.

We can apply these operations which results in

$$L_3L_2L_1Ax = L_3L_2L_1y$$

which will give us a upper triangular matrix. We can then solve and use backwards substitution to get our answer from the upper triangular matrix.

Let us denote this as

$$U = L_3L_2L_1A$$

Thus, we can write everything as

$$Ux = L^{-1}y \Rightarrow LUx = y$$

where U is always an upper triangular matrix and the L is always a lower triangular matrix. This only works with elimination methods and not row swaps or replacements. This can be easily solvable by having

$$LUx = y \Rightarrow Lz = y$$

solve for z , do a backward substitution for U and we can easily get the coefficients for x

Lecture 3

Lecture 4: 10:10

Jan 30 10:10

Could not attend. Check lecture slides instead.

4 Feb 2025

Lecture 5: 10:10

1.1 Linear Iterative Schemes for Linear Systems

6 Feb 2025

Definition 1.1.1. Define the error at step k to be

$$e^k = x^k - x \Rightarrow e^{k+1} = Be^k$$

$$\Rightarrow e^k = B^k e^0$$

We see if B is greater than 1, we are amplifying the noise of the answer. Thus, we want to use some method to get the size of the matrix $I - A$ to be less than 1

1.1.1 Jacobi Iteration

For the Jacobi Iteration we can write that

$$A = P - N$$

$$Ax = y \Rightarrow Px = Nx + y$$

Remark. We have rewritten the equation in this form but is not in the form $x = Bx + y$. We can write instead

$$X = P^{-1}Nx + P^{-1}y$$

We can now perform the iteration here which makes the calculations much easier.

Recall that the Jacobi is going to consist of

$$P = D_A, N = L_A + U_A$$

Remark. Selecting $P = D_A$ is useful because the inverse of P is simply

$$\begin{pmatrix} \frac{1}{p_1} & & \\ & \frac{1}{p_2} & \\ & & \frac{1}{p_n} \end{pmatrix}$$

Thus given x_0 ,

$$x^{k+1} = D_A^{-1}(L_A + U_A)x^k + D_A^{-1}y, k \geq 0$$

Note that if we have the diagonal to be 0 for one part, the method is not possible. However, it is nice to see when the diagonal is very large.

In component form, we can write this as

$$\begin{aligned} x_1^{k+1} &= \frac{y_1 - \sum_{j \neq 1} a_{1j} x_j^k}{a_{11}} \\ &\vdots \\ x_i^{k+1} &= \frac{y_i - \sum_{j \neq i} a_{ij} x_j^k}{a_{ii}} \\ &\vdots \\ x_n^{k+1} &= \frac{y_n - \sum_{j \neq n} a_{nj} x_j^k}{a_{nn}} \end{aligned}$$

Stopping Criteria We have to designate a stopping criteria for linear iterations. One of the methods is using a maximum amount of iterations and the other method is using the relative size of the residual. This is defined as the size of $y - Ax^k$ over the initial as

$$\frac{\|y - Ax^k\|_{l_2}}{\|y - Ax^0\|_{l_2}}$$

Lecture 6: 10:10

Could not attend. Check lecture slides instead.

11 Feb 2025

Lecture 7: 10:10

Could not attend. Check lecture slides instead.

13 Feb 2025

Lecture 8: 10:10

Could not attend. Check lecture slides instead.

18 Feb 2025

Lecture 9: 10:10

Could not attend. Check lecture slides instead.

20 Feb 2025

Lecture 10: 10:10

Could not attend. Check lecture slides instead.

25 Feb 2025

Lecture 11: 10:10

Could not attend. Check lecture slides instead.

4 Mar 2025

Lecture 12

6 Mar 10:10

Chapter 2

Interpolation

Consider the concept of interpolation where we are using extrapolation and generalization. Suppose we had a neural network where we have fitted it with data point $D(t)$. We are asking how general the neural network can be such that we can fit our data.

Definition 2.0.1. Interpolant: Given a discrete set of values y_i at locations x_i an interpolant is a piecewise continuous function $f(x)$ that passes through the data such that

$$f(x_i) = y_i \quad 1 \leq i \leq n$$

After we fit the function we want to evaluate the new data points. Notice that there is a factor that we can't control for such that we don't know the form of $D(t)$ where $D(t) = P_n t^n + P_{n-1} t^{n-1} \dots + P_1$

Example. Suppose we had data points that looked like a linear line. Then we can guess the form

$$D(t) = a + bt$$

Given our plots we have a linear set of equations

$$a + bt_1 = D_1$$

$$a + bt_2 = D_2$$

...

Example. If we want a quadratic interpolation we need three data points

Let $P_2(x) = p_0 + p_1 x + p_2 x^2$, then we have the requirements

$$\sum_{i=1}^3 p_0 + p_1 x_i + p_2 x_i^2 = y_i$$

which can be put into matrix form and solved which will give a form in terms of fractions.

In general, given $n + 1$ data points we can generate an n -th order polynomial of the form

$$\begin{pmatrix} 1 & x_0 & \dots & x_0^n \\ 1 & x_1 & \dots & x_1^n \\ \vdots & \vdots & \ddots & \vdots \\ 1 & & \dots & x_n^n \end{pmatrix} \begin{pmatrix} p_0 \\ p_1 \\ \vdots \\ p_n \end{pmatrix} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

$$Ap = y$$

Theorem 2.0.1. Weierstrass Approximation Theorem: Let $f \in \mathcal{C}([a, b])$ be given. For $\forall \epsilon > 0, \exists$ a polynomial $P(x)$ such that

$$|P(x) - f(x)| \leq \epsilon, \quad \forall x \in [a, b]$$

2.1 Lagrange Interpolation

Theorem 2.1.1. Suppose we had $n + 1$ distinct points. Then there exists a unique polynomial of degree n such that $P(x)$ pass through these points. We must have

$$P(x) = \sum_{k=0}^n f(x_k) L_{n,k}(x), \quad L_{n,k}(x) = \prod_{i=0, i \neq k}^n \frac{x - x_i}{x_k - x_i}$$

where L are the Lagrange interpolating polynomials at node x_k

Remark. Recall the coefficients of a nice structure in terms of fractions. This structure can go to arbitrary orders which lead to the lagrange interpolation. Thus we get the following form nice form where

$$P(x) = \sum_{k=0, x_n \neq x_k}^n f(x_k) \frac{x - x_0}{x_k - x_0} \frac{x - x_1}{x_k - x_1} \cdots \frac{x - x_n}{x_k - x_n}$$

This term is given as $L_{n,k}(x)$. Thus we get the form

$$P(x) = y_0 L_{n,0}(x) + y_1 L_{n,1}(x) + \cdots + y_n L_{n,n}(x)$$

Observe that

$$L_{n,k}(x_j) = \delta_{kj} = \begin{cases} 1, & k = j; \\ 0, & k \neq j; \end{cases}$$

Theorem 2.1.2. Error of Lagrange Interpolation: If $f \in \mathcal{C}^{n+1}(pa, b)$, then we have that

$$f(x) = \sum_{k=0}^n f(x_k) L_{n,k}(x) + \frac{f^{(n+1)}(\eta)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

where the second term is defined to be the error. We can see a taylor expansion where this is the residual terms.

Lecture 13: 10:10

2.2 Lagrange Interpolation Proof

11 Mar 2025

Proof. The Lagrange polynomial $L_{nk}(x)$ is given as

$$L_{nk}(x) = \prod_{i=0, i \neq k}^n \frac{x - x_i}{x_k - x_i}$$

If $L_{nk}(x_j) = \delta_{kj}$ then if $k = j$ the whole expression cancels to be one. And 0 if otherwise. Let us now show that

$$P(x_j) = \sum_{k=0}^n f(x_k) L_{nk}(x_j) = f(x_j), \quad 0 \leq j \leq n$$

which we can do easily. Since $P(x)$ passes through all nodes, it must be a unique answer since for a function s.t $Q(x_j) = y_j$ must be that $P(x_j) - Q(x_j)$ which implies uniqueness. Since both are n

degree, by the fundamental theorem of algebra it must be that with n real roots they are the same. ■

Example. Below are examples of graphs generated by Lagrange Interpolation

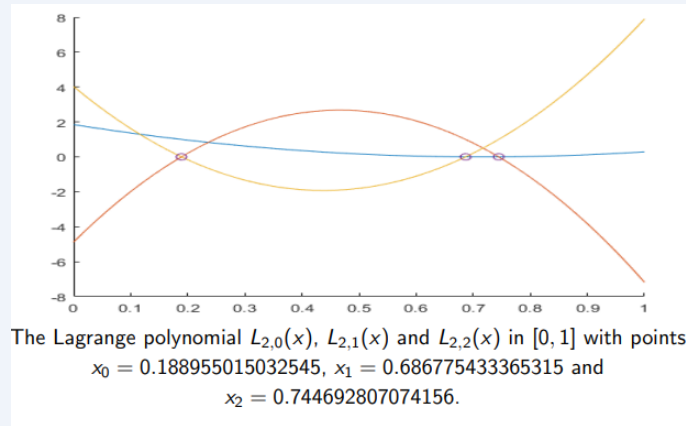


Figure 2.1

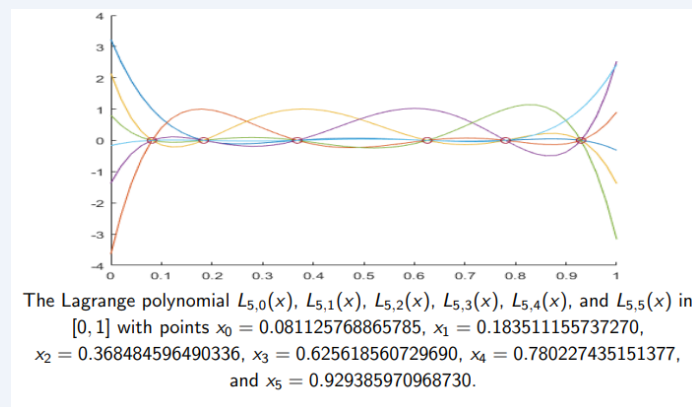


Figure 2.2

Remark. Lagrange interpolation is an explicit formula without any additional computation. For example, a interpolation from two points has the form

$$P_1(x) = p_0 + p_1x$$

$$P_1(x) = y_0 \frac{y_1 y_0}{x_1 - x_0} (x - x_0)$$

2.3 Hermite Interpolation

Definition 2.3.1. Osculatory Polynomial Approximation: Suppose we had $n + 1$ distinct points on a closed interval with a set of $n + 1$ arbitrary integers taking $m \equiv \max(m_0, m_1, \dots, m_n)$. The osculating polynomial approximation is a polynomial $P(x)$ s.t.

$$\frac{d^{k_i} P}{dx^{k_i}}(x_i) = \frac{d^{k_i} f}{dx^{k_i}}(x_i), \quad \forall 0 \leq k_i \leq m$$

We can understand two approximations: taylor and lagrange polynomial approximation where Taylor is

given by

$$P(x) = \sum_{k=0}^{m_0} \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k$$

For the Taylor approximation, we are matching the derivatives of the function f to an order up to m_0 at x_0 . For the Lagrange approximation we are matching all $n + 1$ function values rather the values of the derivatives.

Definition 2.3.2. Hermite Approximation: When $m_i = 1$ for all $0 \leq i \leq n$ we wish to match the function values there as well as the first derivatives such that

$$P(x_i) = f(x_i), \quad P'(x_i) = f'(x_i)$$

Theorem 2.3.1. Let $f \in C^1$ with points. There exists a unique polynomial that it matches the Hermite Approximation conditions with a degree of at most $2n + 1$ where

$$H_{2n+1}(x) = \sum_{j=0}^n f(x_j) H_j(x) + \sum_{j=0}^n f'(x_j) \hat{H}_j(x)$$

where

$$H_j(x) = [1 - 2(x - x_j)L'_{nj}(x_j)] L_{nj}^2(x), \quad \hat{H}_j(x) = (x - x_j)L_{nj}^2(x)$$

Theorem 2.3.2. If $f \in C^{2n+2}$ then

$$f(x) = H_{2n+1}(x) + \frac{f^{(2n+2)}(\zeta)}{(2n+2)!} \prod_{i=0}^n (x - x_i)^2$$

Lecture 14: 10:10

2.4 Spline Interpolation

13 Mar 2025

Definition 2.4.1. Piecewise Polynomial Approximation: The lagrange and hermite are global interpolation schemes where they work for all interpolation nodes. We consider local alternatives

Definition 2.4.2. Piecewise Linear Interpolation: Constructs a linear interpolation between each point $[x_i, x_{i+1}]$ where each subinterval is a Lagrange interpolation

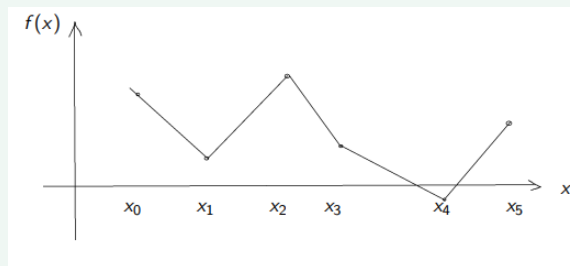


Figure 2.3

Definition 2.4.3. Piecewise Hermite Interpolation: Same as PLI but using a Hermite interpolation s.t. the polynomial is of degree 3.

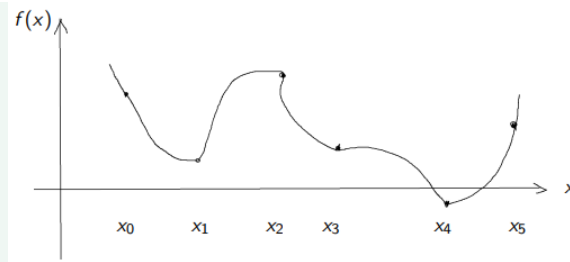


Figure 2.4

What we wish for for a local interpolation has the following properties

1. Smooth and differentiable at nodes
2. One does not need the derivative to approximate $f'(x_i)$

What we wish to find is the cubic spline interpolation.

Definition 2.4.4. Cubic Spline Interpolation: Given a function f defined with given nodes. The interpolant $S(x)$ for $f(x)$ satisfies the following conditions:

1. $S(x)$ is a cubic polynomial
2. $S_j(x_j) = f(x_j)$ and such for all points
3. $S_{j+1}(x_{j+1}) = S_j(x_{j+1}) = f(x_{j+1})$
4. $S'_j = S'_{j+1}$
5. $S''_j = S''_{j+1}$
6. The boundary conditions must be satisfied where either $S''(x_0) = S''(x_n) = 0$ or we must have that $S'(x_0) = f'(x_0)$ and such for all other S'

Example. Suppose we have an example where we have the subintervals given as $[1, 2]$ and $[2, 3]$

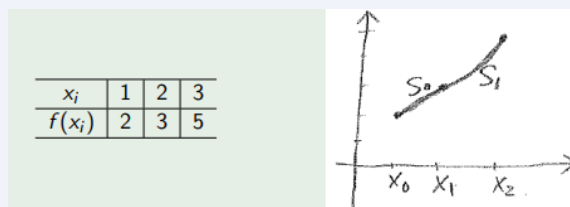


Figure 2.5

which are given by the equation of the form

$$S_i(x) = a_i + b_i x + c_i x^2 + d_i x^3$$

where the index i denotes the equation. Given 8 unknowns from the two subintervals we have 8 equations to solve.

1. Match $S_0(x_0) = f(x_0)$
2. Match $S_0(x_1) = f(x_1)$
3. Next match the second line segment S_1 to the B.C
4. Match the first derivatives
5. Match the second derivatives

6. Use the natural boundary condition at x_0 and x_2 where the second derivatives must be 0

Satisfying these 8 conditions we can solve which gives us the cubic spline interpolant

Theorem 2.4.1. Uniqueness of Natural Cubic Spline Interpolant: If f is defined for an interval $[a, b]$ with points inside, f has a unique spline interpolant s.t. it satisfies the natural B.C. s.t. $S'''(a) = S'''(b) = 0$

From this we can define a general procedure to solve for the cubic spline. For notation, define $I_j = [x_j, x_{j+1}]$ for the j th interval. Then we have the segment as

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3$$

Additionally define $h_j = x_{j+1} - x_j$ The cubic spline is then written as

$$S(x) = \begin{cases} S_0(x), x \in I_0; \\ \dots \\ S_{n-1}(x), x \in I_{n-1}; \end{cases}$$

1. Fit the function on the nodes s.t. $S_j(x_j) = f(x_j)$ where we are matching terms $a_j = f(x_j)$
2. Match interpolation on interior nodes for $S_j(x_{j+1}) = S_{j+1}(x_{j+1})$ and so on s.t. we get the form $a_j + b_j h_j + c_j h_j^2 + d_j h_j^3 = a_{j+1}$
3. Match the derivatives of the interior nodes where we must have $S'_j(x_{j+1}) = S'_{j+1}(x_{j+1})$ which gives the form

$$b_j + 2c_j h_j + 3d_j h_j^2 = b_{j+1}$$

4. Match the second order derivatives at the interpolation nodes s.t. $S''_j(x_{j+1}) = S''_{j+1}(x_{j+1})$ which gives

$$c_j + 3d_j h_j = c_{j+1}$$

5. Apply the natural boundary conditions s.t. $S''(x_0) = 0 \Rightarrow c_0 = 0$
6. Apply the natural B.C. on the left s.t. $S''_{n-1}(x_n) = 0$

$$x_n = 0 \quad c_{n-1} + 3d_{n-1}h_{n-1} = 0$$

Solve the coefficients to get that

$$d_j = \frac{1}{3h_j} (c_{j+1} - c_j)$$

$$a_{j+1} = a_j + b_j h_j + \frac{h_j^2}{3} (2c_j + c_{j+1})$$

$$b_{j+1} = b_j + h_j (c_j + c_{j+1})$$

Continuing the process, we can solve for all the necessary coefficients needed given the conditions solved.

Remark.

1. We only find coefficients for $S_j(0 \leq j \leq n-1)$ or the set $[a, b, c, d]$. the coefficient $c_0 = 0$ is given by B.C.
2. Introducing extra variables isn't needed for the output of the algorithm, e.g. $a_n \equiv f(x_n), b_n = S'(x_n), c_n = \frac{1}{2}S''(x_n)$
3. Note that in the Natural B.C. case where $c_n = 0$, b_n is undetermined but can be determined if using the clamped B.C.

The algorithm is given as shown below:

Algorithm 2.1: Natural Cubic Spline

Input: Integer n , arrays $x[0 : n]$, $f[0 : n]$

- 1 **Initialize:**
- 2 $a[0 : n] \leftarrow 0$, $h[0 : n - 1] \leftarrow 0$, $y[0 : n] \leftarrow 0$,
- 3 $b[0 : n - 1] \leftarrow 0$, $c[0 : n] \leftarrow 0$, $d[0 : n - 1] \leftarrow 0$;
- 4 **for** $j = 0$ **to** n **do**
- 5 $a[j] \leftarrow f[j]$;
- 6 **for** $j = 0$ **to** $n - 1$ **do**
- 7 $h[j] \leftarrow x[j + 1] - x[j]$;
- 8 **for** $j = 1$ **to** $n - 1$ **do**
- 9 $y[j] \leftarrow \frac{3}{h[j]}(a[j + 1] - a[j]) - \frac{3}{h[j - 1]}(a[j] - a[j - 1])$;
- 10 **Initialize matrix** $A[0 : n, 0 : n] \leftarrow 0$, with $A[0, 0] \leftarrow 1$ and $A[n, n] \leftarrow 1$;
- 11 **for** $j = 1$ **to** $n - 1$ **do**
- 12 $A[j, j - 1] \leftarrow h[j - 1]$;
- 13 $A[j, j] \leftarrow 2(h[j - 1] + h[j])$;
- 14 $A[j, j + 1] \leftarrow h[j]$;
- 15 $c \leftarrow \text{LinSolve}(A, y)$;
- 16 **for** $j = 0$ **to** $n - 1$ **do**
- 17 $d[j] \leftarrow \frac{1}{3h[j]}(c[j + 1] - c[j])$;
- 18 **for** $j = 0$ **to** $n - 1$ **do**
- 19 $b[j] \leftarrow \frac{1}{h[j]}(a[j + 1] - a[j]) - \frac{h[j]}{3}(2c[j] + c[j + 1])$;
- 20 **return** a, b, c, d ;

Chapter 3

Numerical Differentiation

Lecture 16: 10:10

3.1 Definitions of Differentiation

25 Mar 2025

Definition 3.1.1. We know for the definition of a derivative is given as Forward Difference:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Backwards Difference:

$$\lim_{h \rightarrow 0} \frac{f(x) - f(x-h)}{h}$$

Central Difference

$$\lim_{h \rightarrow 0} \frac{f(x+h) - f(x-h)}{2h}$$

We define the following wherefor numerical differentiation that h is small. Note the above definitions are going to be different types of numerical differentiations. To check which gives the best approximation, we can Taylor expand s.t

$$f(x+h) = f(x) + f'(x)h + \frac{1}{2}f''(x)h^2 + \dots$$

$$f(x-h) = f(x) + f'(x)(-h) + \frac{1}{2}f''(x)h^2$$

$$\Rightarrow \frac{f(x+h) - f(x)}{h} = f'(x) + \frac{1}{2}f''(x)h$$

$$\frac{f(x) - f(x-h)}{h} = f'(x) - \frac{1}{2}f''(x)h$$

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{1}{3!}f'''(x)h^2$$

where we see the central difference gives a smaller error where $h^2 \ll 1$ thus the error term shows up only when f''' is very large.

3.2 General Approach $n+1$ Point Formula

Assume we want to approximate $n+1$ points around x to approximate the derivative $f'(x)$. The Lagrange polynomial approximation is given as

$$f(x) = \sum_{k=0}^n f(x_k) L_{nk}(x) + \frac{f^{(n+1)}(\zeta)}{(n+1)!} \prod_{k=0}^n (x - x_k)$$

We can simply differentiate to have the new expression as

$$f'(x) = \sum_{k=0}^n f(x_k) L'_{nk}(x) + \frac{f^{(n+1)}(\zeta)}{(n+1)!} \left[\prod_{k=0}^n (x - x_k) \right]'$$

This is the generic method of differentiation from interpolation which the second term represents the error term.

Remark. If we naively use this method for points spread apart, the second error term ϵ will be very large. This is the case for the Taylor expansion too where the points are very spread apart.

For the general formulation to find the derivatives at x_j we simply plug in to the Lagrange interpolation and take its derivative. We then ask the question of whether or not we recover the finite difference approximation for the setup of x_0, x_1, x_2 .

3.2.1 Three and Five Point Formula

Consider three points difference scheme to approximate the derivative. We can rewrite this in terms of x and h such that they are spatially selected to be uniform where we can plug in for the points of $x - h$, x , and $x + h$. If we calculate the Lagrangian interpolation our result for a uniform mesh is given.

$$f'(x_0) = \frac{1}{2h} [-3f(x_0) + 4f(x_1) - f(x_2)] + \frac{f^{(3)}(\zeta_0)}{3!} h^2$$

$$f'(x_1) = \frac{1}{2h} [-f(x_0) + f(x_2)] + \frac{f'''(\zeta_1)}{3!} h^2$$

$$f'(x_2) = \frac{1}{2h} [f(x_0) - 4f(x_1) + 3f(x_2)] + \frac{f'''(\zeta_2)}{3!} h^2$$

Similarly there exists a five point formula that does the same thing with a higher order accuracy.

Chapter 4

Numerical Differentiation

Lecture 15: 10:10

4.1 Definitions of Differentiation

25 Mar 2025

Definition 4.1.1. We know for the definition of a derivative is given as Forward Difference:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Backwards Difference:

$$\lim_{h \rightarrow 0} \frac{f(x) - f(x-h)}{h}$$

Central Difference

$$\lim_{h \rightarrow 0} \frac{f(x+h) - f(x-h)}{2h}$$

We define the following wherefor numerical differentiation that h is small. Note the above definitions are going to be different types of numerical differentiations. To check which gives the best approximation, we can Taylor expand s.t

$$f(x+h) = f(x) + f'(x)h + \frac{1}{2}f''(x)h^2 + \dots$$

$$f(x-h) = f(x) + f'(x)(-h) + \frac{1}{2}f''(x)h^2$$

$$\Rightarrow \frac{f(x+h) - f(x)}{h} = f'(x) + \frac{1}{2}f''(x)h$$

$$\frac{f(x) - f(x-h)}{h} = f'(x) - \frac{1}{2}f''(x)h$$

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{1}{3!}f'''(x)h^2$$

where we see the central difference gives a smaller error where $h^2 \ll 1$ thus the error term shows up only when f''' is very large.

4.2 General Approach $n+1$ Point Formula

Assume we want to approximate $n+1$ points around x to approximate the derivative $f'(x)$. The Lagrange polynomial approximation is given as

$$f(x) = \sum_{k=0}^n f(x_k)L_{nk}(x) + \frac{f^{(n+1)}(\zeta)}{(n+1)!} \prod_{k=0}^n (x-x_k)$$

We can simply differentiate to have the new expression as

$$f'(x) = \sum_{k=0}^n f(x_k) L'_{nk}(x) + \frac{f^{(n+1)}(\zeta)}{(n+1)!} \left[\prod_{k=0}^n (x - x_k) \right]'$$

This is the generic method of differentiation from interpolation which the second term represents the error term.

Remark. If we naively use this method for points spread apart, the second error term ϵ will be very large. This is the case for the Taylor expansion too where the points are very spread apart.

For the general formulation to find the derivatives at x_j we simply plug in to the Lagrange interpolation and take its derivative. We then ask the question of whether or not we recover the finite difference approximation for the setup of x_0, x_1, x_2 .

4.2.1 Three and Five Point Formula

Consider three points difference scheme to approximate the derivative. We can rewrite this in terms of x and h such that they are spatially selected to be uniform where we can plug in for the points of $x - h$, x , and $x + h$. If we calculate the Lagrangian interpolation our result for a uniform mesh is given.

$$f'(x_0) = \frac{1}{2h} [-3f(x_0) + 4f(x_1) - f(x_2)] + \frac{f^{(3)}(\zeta_0)}{3!} h^2$$

$$f'(x_1) = \frac{1}{2h} [-f(x_0) + f(x_2)] + \frac{f'''(\zeta_1)}{3!} h^2$$

$$f'(x_2) = \frac{1}{2h} [f(x_0) - 4f(x_1) + 3f(x_2)] + \frac{f'''(\zeta_2)}{3!} h^2$$

Similarly there exists a five point formula that does the same thing with a higher order accuracy.

Lecture 17: 10:10

4.3 Richardson's Extrapolation

1 Apr 2025

Definition 4.3.1. Richardson extrapolation is used to construct higher order approximations for lower order approximations

For the setup, we approximate ff with $f_1(h)$ of the form

$$f = f_1(h)K_1h + k_2h^2 + \dots$$

where the K s don't depend on x . With this scheme, we can construct higher order approximations. For example given that

$$f = f_1\left(\frac{h}{2}\right) + K_1\left(\frac{h}{2}\right) + K_2\left(\left(\frac{h}{2}\right)^2\right)$$

Then subtracting twice of this and the second we have

$$\begin{aligned} f &= 2f_1\left(\frac{h}{2}\right) - f_1(h) - K^2\frac{h^2}{2} + \mathcal{O}(h^3) \\ 2f_1\left(\frac{h}{2}\right) - f_1(h) &= 4\frac{f(x + \frac{h}{2}) - f(x)}{h} - \frac{f(x + h) - f(x)}{h} \\ &= \frac{4f(x + \frac{h}{2}) - 3f(x) - f(x + h)}{h} \\ &= \frac{1}{h} \left(-f(x + h) + 4f(x + \frac{h}{2}) - 3f(x) \right) \end{aligned}$$